



Python & Virtual Environments

Setting up a virtual environment is not strictly necessary, however it is good practice and a useful habit to build. It essentially creates a little isolated "bubble" for your project, ensuring that the packages and Python versions you use for one project don't interfere with your other projects.

This guide walks you through how to set up a conda environment and use it in VS Code. (Note: there many other ways you can set up a Virtual Environment that work just as well – if you prefer to follow a different guide you have found feel free to do so!)

Step 1: Open Your Project in VS Code

1. Open **VS Code**.
2. Go to **File > Open Folder...** (or **Open...** on Mac) and select the folder where you want your Python project to live.

Step 2: Open the Integrated Terminal

You'll need a place to type your conda commands. You can use the built-in Terminal of VS Code

1. Go to the top menu and click **Terminal > New Terminal**.
2. A panel will pop up at the bottom of your screen.

(Note: Depending on your default settings, this terminal might open as PowerShell, Command Prompt, or bash. As long as Anaconda/miniconda was added to your system path during installation, conda commands will work here!

Alternatively, you can open the conda terminal outside of VS Code, simply search for conda (or anaconda) in your devices Search Bar, and do the following steps in that terminal.)

Step 3: Create Your Conda Environment

1. In the terminal, type the following command and press **Enter**:

```
conda create --name my_first_env python=3.11
```

Explanation:

- `conda create`: Tells conda to make a new environment.
 - `--name my_first_env`: This names your environment "my_first_env" (you can change this to whatever makes sense for your project.
 - `python=3.11`: Tells conda which version of Python to install inside this environment (3.11. was the newest when this guide was written).
2. Conda will calculate what it needs to install and ask: `Proceed ([y]/n)?`.
 3. Type **y** and press **Enter**.

Step 4: Activate Your Environment

Now you need to enter the environment.

1. In the same terminal, type: `conda activate my_first_env` (Remember to swap "my_first_env" with whatever you named it!)
2. You will know it worked because your terminal prompt will change. You should now see `(my_first_env)` at the very beginning of the typing line.

Step 5: Link the Environment to VS Code

If you have been working in a separate conda/anaconda terminal so far, please open VS Code again, in the folder specified at the beginning.

1. Open the **Command Palette** by pressing:
 - **Windows/Linux:** Ctrl + Shift + P
 - **Mac:** Cmd + Shift + P
2. Type **Python: Select Interpreter** and click on the option when it appears in the dropdown.
3. A list of available Python installations will show up. Look for the one that says **Python 3.11.x ('my_first_env': conda)** and click it. *(If you don't see it immediately, click the "refresh" icon in the top right of that little menu).*

You're All Set! 🐍

To verify everything is working smoothly, look at the bottom right corner of your VS Code window (the status bar). You should see your Python version and the name of your conda environment displayed there.

Now, any time you open a `.py` file and click the "Play" button in the top right, it will use your environment!

Using `venv` (The Built-in Alternative to Conda)

While Conda is an excellent third-party tool (especially for data science), Python actually comes with its own built-in virtual environment creator called `venv`.

The core concepts are exactly the same as Conda—you create the environment, activate it, and link it to VS Code. The only difference is the commands you type into the VS Code terminal.

Here is how you use `venv`:

1. Create the Environment Open your VS Code terminal (inside your project folder) and type: `python -m venv my_venv` (*This tells Python to run the `venv` module and create a new folder named `my_venv` right inside your project directory.*)

2. Activate the Environment Unlike Conda, `venv` activation commands are different depending on your operating system:

- **Windows (Command Prompt):** `my_venv\Scripts\activate.bat`
- **Windows (PowerShell):** `my_venv\Scripts\Activate.ps1`
- **Mac / Linux:** `source my_venv/bin/activate`

3. Link it to VS Code Just like with Conda, press `Ctrl + Shift + P` (or `Cmd + Shift + P` on Mac), type **Python: Select Interpreter**, and select the one that points to your new `my_venv` folder.

Why Should You Use Virtual Environments?

Imagine your computer has one giant, global toolbox (your main Python installation). Every time you start a new project, you throw new tools (packages like `pandas`, `django`, or `requests`) into that same box. Eventually, things get messy. Virtual environments solve this by giving each project its own, completely separate toolbox.

- **Avoiding Version conflict:** Let's say Project A is older app and requires version 1.0 of a package. Project B is a brand-new and needs version 2.0 of that exact same package. You cannot have both versions installed in the same global toolbox at the same time. Virtual environments allow Project A and Project B to exist on the same computer with different package versions.
- **Easy Reproducibility:** If you want to share your code with a friend or upload it to a server, they need to know exactly which packages you used. A virtual environment allows you to easily generate a list of *only* the packages used in that specific project.